

Implementeringsstrategi: Övergång till ACME:s Domänneutrala Arkitektur

Denna strategi definierar ramverket för att transformera osäkra generativa arbetsflöden till deterministiska, granskningsbara och domänstyrda system genom ACME (Adaptive Context Memory Engine). Som senior AI-arkitekt fastställer jag härmed att målet är en strikt separation mellan generativ slutledning och domänlogisk auktoritet, där språkmodellen degraderas från beslutsfattare till kandidatgenerator.

1. Strategiskt ramverk: Från traditionella AI-lager till ACME

Skiftet från traditionella LLM-integrationer till ACME:s modulära arkitektur är en nödvändighet för att uppnå teknisk integritet. I traditionella system är promptar, affärslogik och tillstånd sammanflätade, vilket gör dem omöjliga att granska. ACME eliminerar detta genom att isolera domänlogiken i moduler medan kärnan (Core) förblir strikt domänneutral. Inga begrepp som "vittne", "hypotes" eller "kapitel" tillåts existera i kärnan.

Paradigmsskifte: Jämförelse av arkitekturprinciper

Dimension, Traditionell AI-lager, ACME Domänneutral Arkitektur
Truth Authority, Språkmodellen (LLM), Domänpolicy & StateEngine
State Mutation, Implicit/Ogenomskinlig uppdatering, Deterministisk Reducer & Invarianter
Failure Handling, Opaque Retries, Revisioned Unit of Work (Atomicitet)
Durable Resume, Omtag från start, Resume utan extra leverantörsanrop
Evidence Integrity, Ofta saknad/Ad-hoc, Operation Digest (Prepared Commit)

Separation av ansvar: De tre portarna

För att garantera systemets integritet passerar varje kandidat genom tre strikta portar i en atomisk transaktionskedja:

1. **Validate:** Teknisk och semantisk validering av kandidaten mot strikta kontrakt.
2. **Interpret:** Domänmodulen projicerar den validerade kandidaten till minneskandidater och explicita intentioner (stateIntent).
3. **Commit:** Domänens policyer appliceras, invarianser verifieras och hela aggregatet skrivs atomiskt som en **Revisioned Unit of Work**. Denna separation minskar teknisk skuld genom att frikoppla domänlogik från leverantörs-SDK:er, vilket möjliggör leverantörsbyten utan att påverka affärsregler eller evidensens giltighet.

2. Definition av PromptContracts: Grindvakter för datakvalitet

PromptContracts utgör systemets första defensiva lager. De tvingar fram en maskinläsbar struktur på den generativa kandidaten innan den tillåts interagera med domänlogiken.

Strukturering via ResponsePipeline

För att eliminera "silent repairs" och hallucinerad struktur följer ResponsePipeline dessa normativa steg:

1. **Schema-validering:** Utdata valideras mot Zod-scheman. Systemet vägrar uttryckligen schema-coercion (automatisk typomvandling); alla avvikelser i datatyp leder till omedelbar exekveringsvägran.
2. **Runtime-validering:** Tekniska kontroller av JSON-integritet och extraktion.

3. **Semantisk kontroll:** Validering av utdata sker mot **detached och deeply frozen** indata. Detta säkerställer att valideringen utförs mot en oföränderlig källa och förhindrar mutation under kontrollsteget.

Input-bound validering och immutabilitet

Genom att validera indata *innan* utdata inspekteras förhindras modellen från att uppfinna instruktioner som strider mot givna ramvillkor. Varje kontrakt är immutabelt; varje ändring kräver en ny version för att bibehålla replay-stabilitet. Detta skapar en obruten kedja från teknisk validering till domänpolicy.

3. Implementering av Domänpolicyer: Att koda betydelse

Domänmodulerna äger betydelsen av information genom MemoryEngine och StateEngine. Medan kärnan hanterar mekaniken, äger modulen sanningen.

Research-policyn: Vetenskaplig korroborering

Research-domänen använder research-proposition-key-1 för att identifiera unika påståenden. En central princip är käll-oberoende:

- **Korroborering:** Ett påstående främjas till "verified" endast när det stöds av distinkta research-source-independence-key-1. Detta förhindrar att data från samma lab eller dataset räknas som oberoende bekräftelse.
- **Contest-modell:** Vid motstridiga källor behålls båda positionerna i minnet för att spegla vetenskaplig osäkerhet, snarare än att tyst skriva över data.

Legal-policyn: Situated Assertions

Till skillnad från Research fokuserar Legal på **situated assertions** (utsagor bundna till kontext). Identitetsnycklar härleds från tre dimensioner:

1. **Who:** Talaren (Speaker).
2. **When:** Tidpunkt (Time).
3. **Where:** Ursprung (Artifact/Locator). Legal-policyn tillämpar en "contest/coexist"-modell. Information raderas aldrig; motsägelser dokumenteras som konflikt-kanter i grafen. Supersession är reserverat för explicita korrigeringar av samma artefakt-linje (t.ex. en rättad transkription).

Deterministisk Identitet

Alla identiteter skapas via acme-cjson-1 och SHA-256 innehållshashing. Detta genererar deterministiska acme-transition-id-1, vilket möjliggör spårbar proveniens från råkälla till kanoniskt tillstånd utan behov av centraliserade ID-generatorer under logikexekvering.

4. ExecutionEngine: Validering och exekveringskontroll

ExecutionEngine garanterar att inga domänbeslut fattas direkt av språkmodellen. Den koordinerar flödet där stateIntent betraktas som icke-betrodd (untrusted) tills den passerat domänens reducer och invarianter.

Exekveringsprotokoll (Sequential Protocol)

1. **Accept:** Validering av request och idempotens.
2. **Reserve:** Reservation i ledgern.

3. **Model Call:** Anrop via gateway.
4. **Validate:** Pipeline-validering (schema/semantik).
5. **Interpret:** Projicering till stateIntent (untrusted).
6. **Prepare:** Anrop av projectState() där deltat skapas och förbereds för en **Compare-and-Swap (CAS)** operation.
7. **Commit:** Atomisk skrivning av Prepared Commit till ledgern.

Resume-funktionalitet och hållbarhet

Vid avbrott efter ett lyckat modellanrop återupptas exekveringen genom att läsa det inspelade svaret från ledgern. Systemet gör **aldrig** ett nytt anrop till leverantören om evidens finns. Om en reservation gjorts men inget svar registrerats, terminaliseras exekveringen som MODEL_UNAVAILABLE för att undvika osäkra antaganden om leverantörens tillstånd.

5. Replay och Evidens: Systemets hållbarhet

Deterministisk replay är fundamentet för systemets granskningsbarhet.

Operation Digest

Kärnan i verifieringen är acme-operation-digest-1. Detta är en innehållshash av hela **Prepared Commit** – det kompletta paketet av indata, modellanrop, minnesbeslut och tillståndsförändringar. Genom att jämföra digesten från en offline-körning med den lagrade evidensen kan en arkitekt verifiera exekveringen utan att belasta live-systemet eller kontakta språkmodeller.

Retention och Kryptering

Systemet stöder tre nivåer av retention. Vid nivån encrypted-payload krypteras känsligt innehåll med AES-256-GCM. Replay är i dessa fall tekniskt omöjligt utan den injicerade nyckeln, vilket binder möjligheten till granskning direkt till organisationens säkerhetspolicy.

6. Färdplan för implementering och beviskriterier

För att bevisa arkitekturens domänneutralitet krävs en sekventiell implementering där samma kärna driver radikalt olika moduler.

Build Order & Proof Criteria

1. **Policy-definition:** Fastställ domänens policyer och PromptContracts (Zod).
 2. **Offline Proof:** Exekvering mot "golden vectors" och mockade gateways.
 3. **Live Integration:** Anslutning av live-gateways med verifierad schema-lowering.
- Beviskriterier för godkänd implementering:**
- **Zero Domain Vocabulary:** Ingen domänspecifik terminologi får finnas i kärnan.
 - **Stable Digest:** Identiska digests vid replay av samma evidens.
 - **Conflict Integrity:** Korrekt hantering av motsägelser enligt domänpolicy (contest/coexist) utan dataförlust.

Sammanfattning: Framtidssäkrad infrastruktur

Denna strategi transformerar AI från en "svart låda" till en robust affärsinfrastruktur. Genom strikt logisk separation och fullständig spårbarhet möter ACME de regulatoriska kraven i **EU**

AI Act (Regulation 2024/1689) gällande transparens och loggning för högrisk-system. ACME är inte bara en motor; det är fundamentet för framtidens juridiskt och tekniskt försvarbara AI-applikationer.